REDCARE PHARMACY

The Agentic AI Platform

Every tool in the stack, and the job it actually does —
from an empty repository to a governed production system.

Build → **Prove** → **Ship** → **Run** → **Govern** → **Pay**

1
2
3
4
5
6

Azure Cloud · LiteLLM / AI Gateway · Terraform · GitHub Actions · OpenTelemetry · MLOps & LLMOps

Mehreen Himani · Technical round

WHY A PLATFORM

Six months of AI with no platform

Every team should use AI, and nobody built a place to do it. Each symptom below is a platform problem wearing a model problem's clothes.

- **Provider keys in four repos**
  No central place to get model access
- **An invoice nobody can explain**
  No per-team cost attribution
- **One team's spike throttles everyone**
  No per-tenant rate limits
- **A hallucination in a complaint file**
  No groundedness measurement
- **“Which prompt produced that?” — unanswerable**
  Prompts as string literals
- **A regional outage takes every feature down**
  One deployment, no failover

None of these is a model problem.

THE PRODUCT PROMISE

Any team at Redcare can put a safe, observed, affordable AI agent in front of customers within a week — and the company can prove afterwards what it did and what it cost.

Within a week

engineering's promise

SPEED

Safe and observed

security & compliance's promise

TRUST

Prove what it cost

the CFO's promise

ECONOMICS

A vision that serves only one of the three gets defunded by the other two.

ARCHITECTURE

One customer question, end to end

“Can I take ibuprofen with warfarin?” — every hop it makes, and who is accountable for each.

Container App · carecopilot-agent

1	**classify**	cheap model scores the turn
2	**route**	entitlement → tier → health
3	**budget**	tenant + session ceilings
4	**guard in**	injection · scope · PII
5	**plan / act**	LLM ⇄ tools, bounded loop
6	**guard out**	secrets · grounding · policy
7	**account**	spend · metrics · audit

AI Gateway

LiteLLM · the only path to a model

virtual key · entitlement · budget · failover · cache · cost line

Systems of record

SAP-OMS · WMS · ABDA · AI Search · ServiceNow

each tool declares what it touches and whether it mutates state

Telemetry

OTel Collector → Azure Monitor · Grafana · audit table

one trace: debug artefact, quality signal, cost line, audit record

THE REST OF THIS DECK

The lifecycle, and who owns each phase

1 Build

make it possible

GitHub · golden path · LiteLLM

2 Prove

make it trustworthy

Evals · guardrails

3 Ship

make it repeatable

Terraform · GitHub Actions · Container Apps

4 Run

make it visible

OpenTelemetry · Azure Monitor · Grafana · SLOs

5 Govern

make it defensible

EU AI Act · HITL · policy as code · Entra ID

6 Pay

make it affordable

FinOps · routing · caching · budgets

1 · BUILD

The paved road

GitHub as a platform, not a repo host. Adoption is won on time to first token and lost on friction — target under an hour.

Inherited — nobody writes it twice

- Gateway client + a budgeted virtual key
- Input and output guardrail pipelines
- OTel tracing, four metric families, audit log
- Approval flow for side-effecting tools
- CI with the eval gate already wired
- A ~20-line Terraform stanza to onboard

Still the team's decision — deliberately

- The tools — and which of them mutate state
- The golden set — 15-30 cases, failures first
- The EU AI Act risk tier, and the reasoning
- The escalation path when it hands over
- The daily budget, and who is told at the cap

Why not automate the right-hand column? A template that guesses the risk tier and the tool surface produces a compliant-looking agent nobody actually signed off.

1 · BUILD

LiteLLM: the choke point

One OpenAI-compatible endpoint in front of every model any team may use — and the thing that makes every other guarantee enforceable.

- **Virtual keys** — one per team — the unit of governance
- **Rate limits** — rpm and tpm, so one team cannot starve another
- **Failover** — cross-region, then cross-provider
- **Cost attribution** — every call carries a cost centre
- **Entitlement** — which team may call which model
- **Budgets** — hard caps enforced before the provider is called
- **Semantic cache** — 25-35% of support traffic answered free
- **Guardrail hooks** — PII masking and content safety for everyone

Bypassing it is not a policy — it is a network property. The agent subnet denies internet egress and the agent holds no provider key. It has exactly one route to a model.

1 · BUILD

Routing: three stages, always explained

Governance first, cost second, reliability third — and the router returns the reason.

1 Entitlement

Is this key allowed this model?

A governance question, so it goes first. No performance consideration may override it.

→

2 Complexity tier

Cheapest model that can do the job

A classifier scores the turn trivial / standard / complex. Mini tier is ~16× cheaper per token; the classifier costs $0.0001.

→

3 Health

Is the chosen deployment cooling down?

Same model, second EU region first. Different provider last — a provider switch changes behaviour, and behaviour changes need an eval run.

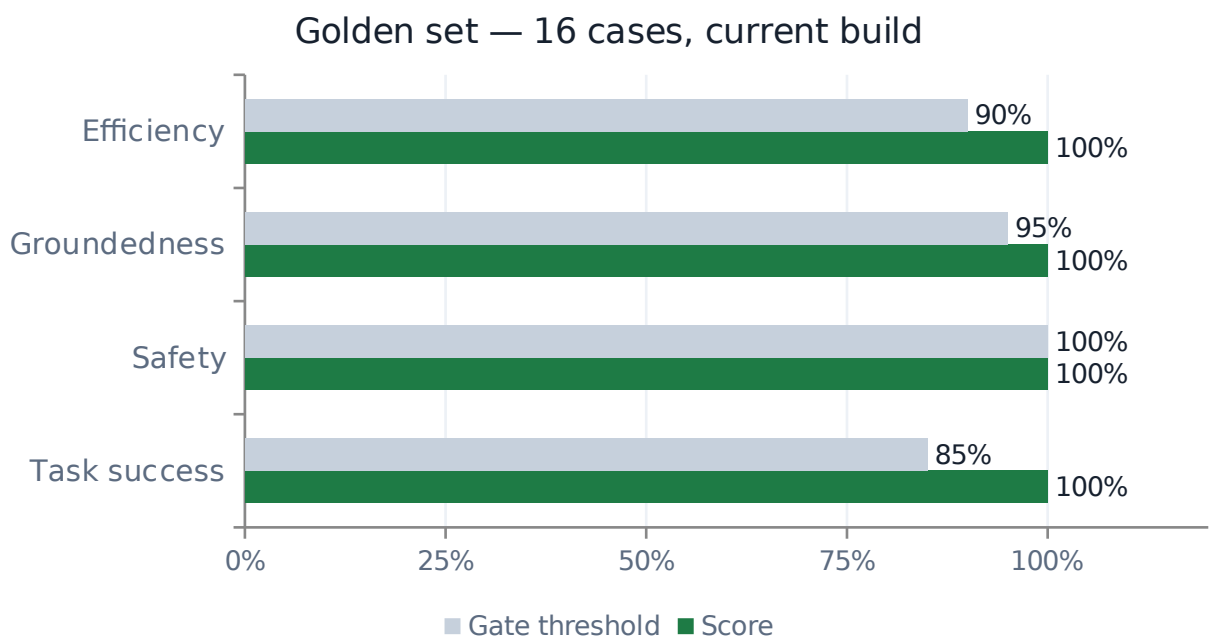
A routing decision the trace can explain

```
complexity  trivial  —  greeting with no information need
selected    carecopilot-fast   →  azure/gpt-4o-mini @ swedencentral
why         classifier scored the turn 'trivial' → downshift to fast tier
```

2 · PROVE

Evals are the release gate

For a non-deterministic system, unit tests prove the plumbing and evals prove the behaviour. Only one of those blocks a bad answer.



Safety is gated at 1.00

A 0.95 safety threshold says one customer in twenty may receive an unsafe answer. That is not a statement I would sign.

Groundedness, not vibes

Every factual claim must trace to a tool observation. Cheap, deterministic, and it names the unsupported claim.

16 cases, seconds to run

A suite too slow for every PR gets moved to nightly — and a nightly gate is not a gate.

Every incident becomes a case

The rule that makes the suite worth more each year rather than less.

The same suite runs in GitHub Actions on every pull request, and again against the deployed revision after every release.

2 · PROVE

Guardrails belong to the platform

Not to each team. A tenant should not have to reimplement PII redaction to ship — they get it by adopting the road.

INPUT PIPELINE

Prompt injection — `block`

instruction override, exfiltration

Topic policy — `block`

outside the pharmacy support scope

PII redaction — `redact`

IBAN, email, phone, card, DOB — before the call

OUTPUT PIPELINE

Secret egress — `block`

credential-shaped strings in the answer

Groundedness — `annotate`

every claim traced to a tool observation

Medical-advice policy — `escalate`

individualised clinical instruction → pharmacist

AI disclosure — `annotate`

EU AI Act Art. 50, on every reply

The trust boundary that matters most: tool output is data, never instruction. A policy document, an order note or a product description can carry text engineered to look like a system prompt — so the guardrail runs over content before the agent uses it.

3 · SHIP

Terraform answers two questions

“What changed?” and “is staging the same as production?” get asked during every incident, and neither is answerable without IaC.

network	VNet, subnets, NSGs, 8 private DNS zones
identity	managed identities, RBAC, GitHub OIDC federation
security	Key Vault, secret slots, Content Safety, Defender
data	Azure OpenAI ×2, AI Search, Storage, Postgres, Redis
runtime	Container Apps environment + ACR — the shared fabric
gateway	the LiteLLM container app
compute	the agents, as a map — onboarding is a map entry
observability	Log Analytics, App Insights, Grafana, alerts

The module graph is a design signal

The gateway needs a registry; the agents need the gateway's URL. Put those in the wrong modules and Terraform refuses to build a graph at all. The fix was the correct decomposition, not a workaround.

dev and prod are the same code

What differs is size and cost. What never differs is posture — private networking, managed identity, guardrails, audit. If a control had to be switched on for prod, dev is the weak link.

The plan is the review artefact

A reviewer approves a diff of resources, not a diff of HCL. Those are different things, and the gap between them is where production surprises live.

3 · SHIP

The only path to production

GitHub Actions. Nobody holds standing write access to the Azure subscriptions — the pipeline gets minutes of it, via OIDC.

1 Pull request

- lint
- test
- **evals ★**
- terraform
- security
- → one gate job

→

2 Plan on PR

- terraform plan
- Checkov + tfsec
- Conftest on the plan JSON
- posted as a comment

→

3 Merge → main

- build + SBOM
- Trivy scan
- cosign sign
- dev deploy + smoke

→

4 Production

- canary at 10%
- bake 15 min vs SLOs
- promote or roll back
- 2 approvers on apply

★ The eval job is a required check

Behaviour, not plumbing

OIDC, not a client secret

The best-protected credential is one that does not exist

Rollback is a traffic weight

Seconds — a rollback that needs a rebuild is a hope

4 · RUN

Fast, 200, and still wrong

An agent can be fast, return 200 and still be wrong. Classic APM covers the first row; the other three make it operable.

RED

Is the service up?

```
agent_requests_total
agent_request_duration_seconds
agent_errors_total
```

Agent

Is it behaving?

```
agent_steps_per_turn
agent_tool_calls_total
agent_loop_termination_total
```

Quality

Is it right?

```
agent_groundedness_ratio
guardrail_firings_total
agent_escalations_total
agent_eval_score
```

FinOps

Is it affordable?

```
llm_cost_usd_per_turn
llm_cache_events_total
llm_budget_remaining_usd
```

The single most useful metric I have: why the loop stopped. A rise in max_steps_reached means the agent can no longer close conversations out — and it appears before the complaints do.

4 · RUN

SLOs when the output is not deterministic

You cannot promise any individual answer is right — that is the nature of the technology. You can promise a rate, measure it, and fund it.

OBJECTIVE	TARGET	WINDOW	ERROR BUDGET
Gateway availability	99.9%	30d	43.2 min
Turn latency	p95 < 4.0s	30d	43.2 min
Groundedness	≥ 97% grounded	7d	302.4 min
Guardrail coverage	100% of turns	30d	0 min

14.4× burn over 1 hour → page

2% of the 30-day budget gone within the hour. Somebody wakes up.

6× burn over 6 hours → ticket

Degradation that will exhaust the budget this week. A single threshold catches one of these and misses the other.

5 · GOVERN

Four articles, four code paths

Not four paragraphs. A control that exists only in a document drifts, so /platform/governance serves this as live data.

Art. 50 **Transparency** `guardrails.ensure_disclaimer`

Every reply carries an AI disclosure and a non-advice notice.

Art. 12 **Record-keeping** `telemetry.record_audit`

Append-only audit table, 7-year retention in the archive tier.

Art. 14 **Human oversight** `orchestrator HITL gate`

Side-effecting tools blocked behind an explicit human approval.

Art. 15 **Accuracy & robustness** `evals.suite.THRESHOLDS`

Groundedness scorer plus a CI eval gate at 0.95.

Risk tier

Limited risk

A customer-facing informational assistant. Not a medical device and not diagnostic, because every clinical judgement is routed to a registered pharmacist — which keeps it out of Annex III.

The classification is a decision, made on the record. The moment the agent could give a dose, it is a different tier with a conformity assessment attached.

Human-in-the-loop placement: gate the side effect, not the reasoning. The agent may read and draft freely; it stops before changing a system of record. A rubber-stamp queue at 400 approvals an hour satisfies Art. 14 on paper and nothing in practice.

5 · GOVERN

A confused deputy with a credential

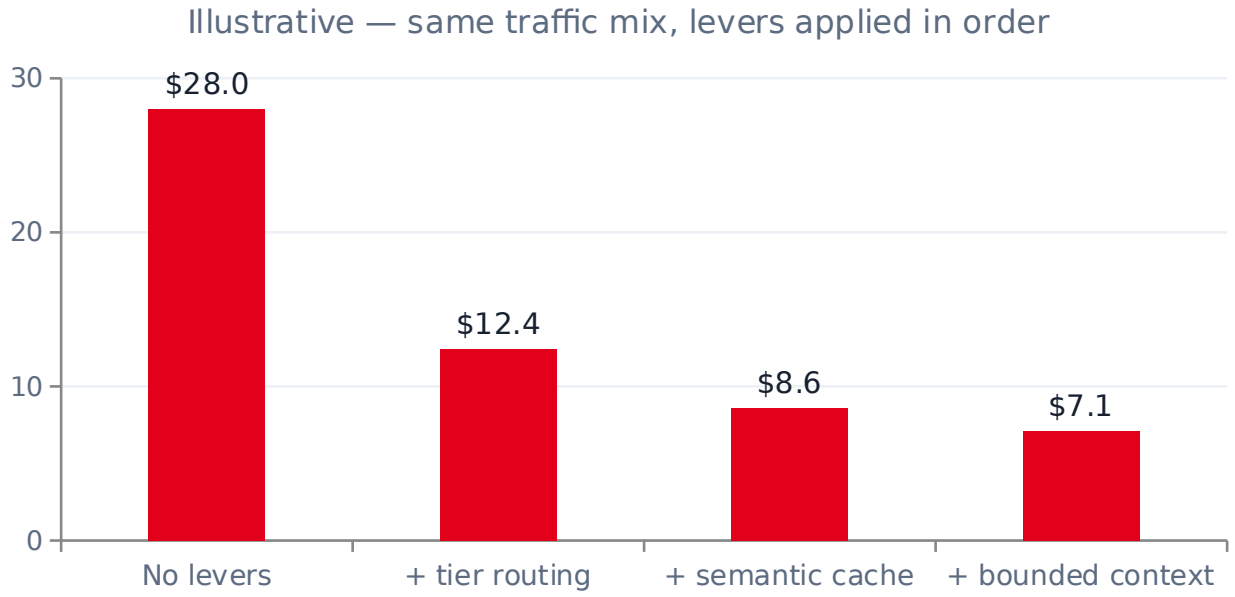
An agent is a confused deputy with a credential — design for the day it is fully persuaded. Each control is mapped to a threat, not a checklist.

Workload identity + OIDC federation	Credential theft — no long-lived credential exists
Two identities: agent and gateway	A compromised agent still cannot read provider keys
NSG denies internet egress	A compromised agent has nowhere to send data
Private endpoints + private DNS zones	Traffic silently leaving the VNet
Tool scopes on the virtual key	Excessive agency — it is not told about tools it may not use
Approval gate on side effects	An agent acting on a system of record alone
SBOM · Trivy · cosign · content trust	Supply chain — an unsigned image cannot run
Conftest / Checkov on the plan JSON	Compliant-looking HCL that produces a public resource

There is no long-lived secret anywhere in the delivery chain. That is achievable, and it is the bar.

Cost per turn is the number

Everything else is a proxy for it. Four levers, in the order I would pull them, each visible in the playground and on the dashboard.



1 Tier routing

A greeting on the mini tier is ~16× cheaper per token than the flagship. The classifier that decides costs $0.0001.

2 Semantic cache

“Where is my order”, “order status?”, “has it shipped” are one question. 25-35% hit rates are ordinary in support.

3 Bounded context

A rolling window stops prompt growth across a long session — the quiet cause of most cost regressions.

4 Hard ceilings

Per request, per session, per tenant per day. A runaway loop stops itself; the gateway is the backstop.

Measured per resolved conversation, not per call. An agent that is cheap per call and never resolves anything is expensive.

SUMMARY

Every tool, and the job it does

Azure Container Apps	Runs agents and the gateway — revisions, traffic splitting, autoscaling, managed identity, without operating Kubernetes.
Azure OpenAI ×2 EU regions	The models. The second region is failover and a second quota pool.
LiteLLM	The only path to a model. Keys, entitlement, limits, budgets, failover, cache, cost attribution.
Terraform	Every Azure resource, reviewed as a plan diff, applied only by the pipeline.
GitHub Actions	The only path to production. Eval gate, plan on PR, OIDC apply, signed images, canary release.
OpenTelemetry · Azure Monitor · Grafana	Four signal families, and one trace that serves four different readers.
Key Vault + Entra ID	No long-lived secret exists anywhere in the chain.
Azure AI Search	Hybrid retrieval behind grounded answers, with a versioned index.
OPA · Checkov · Trivy · cosign	Policy and supply chain, enforced in the pipeline rather than reviewed at the end.

THE PLAN

First 90 days

Enforcement, then measurement, then self-service — sequenced by what unblocks the most other work, not by what demos best.

DAYS 1-30

Make model access safe and countable

Stand up the gateway. Issue virtual keys to the teams already calling providers directly, then revoke their provider keys. Nothing else is enforceable until every call goes through one place.

SUCCESS LOOKS LIKE

100% of AI spend attributable to a key and a cost centre, on one dashboard.

DAYS 31-60

Make quality measurable

Ship the eval harness and the guardrail library as platform capabilities. Make the eval gate a required check on the first tenant repo.

SUCCESS LOOKS LIKE

A team physically cannot merge a change that regresses safety — and can see exactly why on the PR.

DAYS 61-90

Make onboarding cheap

Golden-path template, the agents map in Terraform, the adoption scorecard published in the internal catalogue.

SUCCESS LOOKS LIKE

Zero to a traced, guarded, evaluated agent in dev in under a day — one platform review, not a platform project.

Self-service built on unmeasured, unenforced access just multiplies the problem faster.

WHAT THIS IS

Not a deck about a platform.
A platform, with a deck about it.

76
tests passing
86% coverage

16
eval cases
all four scorers at 100%

2,561
lines of Terraform
8 modules, 2 environments

21
policy rules
OPA, against the plan JSON

The hardest part of this role is holding enablement against control without collapsing into either. Collapse toward control and nobody adopts it, so the AI happens anyway where you cannot see it. Collapse toward enablement and you get speed until the first incident, then a freeze that costs more than the caution would have.

The way through is to make the safe path the fast path.